

CRUD NoSQL com MongoDB

Mapeamento do esquema relacional para um banco de dados NoSQL

**JOSÉ ARTHUR CALIXTO DA ROCHA COSTA
JOSEPH ANTONY DOS SANTOS LEITE
LUAN DANIEL SANTANA COUTO**

[Julho de 2026]

2. Objetivo

A Parte 2 do trabalho prático tem como objetivo o **mapeamento do esquema relacional** (composto pelas tabelas **usuario**, **estudante**, **curso** e **vinculo**) para um banco de dados **NoSQL**, especificamente o **MongoDB**, e a implementação das operações de **CRUD** (Create, Read, Update, Delete) sobre essa nova estrutura.

Esta etapa visa consolidar os conhecimentos sobre bancos de dados não relacionais, explorando suas diferenças fundamentais em relação ao modelo relacional, bem como as estratégias de modelagem de dados orientada a documentos.

3. Pesquisa e escolha do SGBD NoSQL

3.1. Visão Geral dos SGBDs NoSQL

Bancos de dados NoSQL surgiram para atender a demandas que os sistemas relacionais tradicionais não suportavam eficientemente, como alta escalabilidade horizontal, flexibilidade de esquema e tratamento de grandes volumes de dados não estruturados. Os principais tipos são:

Tipo	Exemplos	Características
Chave-Valor	Redis, DynamoDB	Armazenamento simples por chave; alta performance para cache e sessões.
Colunar	Cassandra, HBase	Otimizado para consultas em grandes volumes de dados com muitas colunas.
Orientado a Documentos	MongoDB, CouchDB	Dados armazenados em documentos JSON/BSON; esquema flexível e hierárquico.
Grafos	Neo4j, ArangoDB	Focado em relacionamentos complexos entre entidades (ex: redes sociais).

3.2. Justificativa para a Escolha do MongoDB

Após a pesquisa, optou-se pelo MongoDB pelos seguintes motivos:

1. **Modelagem Natural:** O domínio do problema (universidade) envolve hierarquias claras, como um estudante que possui vários cursos e vínculos. O modelo de documentos JSON reflete essa hierarquia de forma intuitiva, eliminando a necessidade de *joins* complexos.
2. **Flexibilidade de Esquema:** Permite evoluir a estrutura de dados (ex: adicionar novos campos) sem a necessidade de migrações complexas, o que é vantajoso em cenários acadêmicos.
3. **Integração com Spring Boot:** O ecossistema Spring oferece o **Spring Data MongoDB**, que simplifica a implementação do repositório e a interação com o banco, reduzindo o código boilerplate.
4. **Infraestrutura na AWS:** O MongoDB roda na infraestrutura da AWS.
5. **Suporte a Índices:** Oferece índices únicos e compostos, essenciais para garantir restrições de unicidade (ex: CPF e matrícula).

4. Mapeamento Relacional → NoSQL

4.1. Estratégia de Mapeamento

No modelo relacional (Parte 1), tínhamos quatro tabelas normalizadas. Para o MongoDB, adotamos uma abordagem **desnormalizada**, consolidando os dados em um único documento principal (**estudantes**) para otimizar as consultas mais frequentes.

Tabela de Mapeamento:

Modelo Relacional (Tabela)	Modelo NoSQL (MongoDB)	Justificativa
Usuario	Embutido no documento estudante (campos: cpf, nome, dataNascimento, email, telefone, login, senha).	Cada estudante é um usuário. Embutir os dados evita uma segunda consulta para obter as informações pessoais.
Estudante	Documento principal estudante (campos: matricula, mc, anoIngresso, cpf, nome, etc.).	É a entidade central do sistema.
Curso	Subdocumento dentro do array cursos.	Um estudante pode cursar vários cursos ao longo do tempo. O array permite armazenar todo o histórico em um único lugar.
Vinculo	Sub-subdocumento dentro de cada item do array cursos (campo vinculo).	O vínculo (data de entrada, status, data de saída) é uma propriedade específica da relação entre o estudante e aquele curso específico.

4.2. Estrutura do Documento (JSON)

```
{
  "_id": "ObjectId('...')",
  "cpf": "33333333302",
  "nome": "João Silva",
  "dataNascimento": "2000-01-15",
  "email": ["joao@email.com"],
  "telefone": ["79999999999"],
  "login": "joao.silva",
  "senha": "123456",
  "matricula": "999001",
  "mc": 85,
  "anoIngresso": 2025,
  "cursos": [
    {
      "cursoId": "curso_001",
      "nome": "Engenharia de Dados",
      "grau": "Bacharelado",
      "turno": "Noturno",
      "campus": "Aracaju",
      "nivel": "Graduação",
      "vinculo": {
        "dataEntrada": "2025-02-01",
        "status": "Ativo",
        "dataSaida": null
      }
    }
  ]
}
```

4.3. Justificativa para a Desnormalização

A decisão de desnormalizar foi tomada com base nos seguintes fatores:

- **Performance de Leitura:** A consulta mais comum no sistema é recuperar o histórico completo de um estudante. Com um único documento, isso é feito em uma única operação no banco, sem a necessidade de múltiplas consultas ou *lookups*.
- **Atomicidade:** As operações de atualização (ex: adicionar um novo curso ou alterar o status de um vínculo) podem ser feitas atomicamente dentro do mesmo documento.
- **Coerência Contextual:** Os dados do vínculo só fazem sentido no contexto do curso ao qual pertencem. Mantê-los juntos reforça essa coerência.

5. Restrições de Integridade

O MongoDB não possui mecanismos nativos para todas as restrições tradicionais (como chaves estrangeiras). Portanto, as garantias foram implementadas em diferentes camadas da aplicação.

Restrição	Como foi garantida	Local da Validação
Chave Primária (PK)	@Id do Spring Data MongoDB (geração automática do <code>_id</code>).	Banco de dados (MongoDB).
Chave Única (CPF)	@Indexed(unique = true) no campo <code>cpf</code> . Cria um índice único no MongoDB.	Banco de dados (índice).
Chave Única (Matrícula)	@Indexed(unique = true) no campo <code>matricula</code> .	Banco de dados (índice).
NOT NULL / Obrigatoriedade	Uso de anotações @NotNull e @NotBlank nos DTOs (ex: <code>cpf</code> , <code>nome</code> , <code>matricula</code>).	Camada Controller (Spring Validation).
Integridade Referencial (FK)	Validação no Service. Antes de salvar, verificamos se o <code>cursoId</code> existe (se houvesse uma coleção de cursos separada). No modelo atual, como os cursos são embutidos, não há dependência externa	Camada Service (Java).
Domínio (Valores Válidos)	Validação manual no Service para o campo <code>status</code> (ex: "Ativo", "Cancelado", "Graduado", "Formando").	Camada Service (Java).
Formato de Dados	Validações como @Size(max = 7) para <code>matricula</code> , @Past para data de nascimento, etc.	Camada Controller (DTOs).

6. Implementação do CRUD

6.1. Tecnologias Utilizadas

- Java 21
- Spring Boot 3.4.5
- Spring Data MongoDB
- Lombok (redução de boilerplate)
- Maven (gerenciador de dependências)
- MongoDB (hospedagem na AWS EC2)

6.2. Arquitetura da Aplicação

A aplicação segue o padrão **Controller-Service-Repository**:

- **Controller:** Expõe os endpoints REST e recebe as requisições. Converte DTOs para os objetos de domínio.
- **Service:** Contém toda a lógica de negócio, validações, tratamento de exceções e coordenação das operações.
- **Repository:** Interface que estende `MongoRepository`, fornecendo métodos prontos para CRUD e consultas personalizadas (ex: `findByCpf`).
- **Model (Document):** Classes anotadas com `@Document`, `@Id`, `@Indexed`, representando a estrutura do MongoDB.
- **DTOs:** Objetos de transferência de dados para desacoplar a camada de apresentação do modelo interno.

6.3. Arquitetura da Aplicação

Método	Endpoint	Descrição
POST	<code>/api/nosql/estudantes</code>	Cria um novo estudante com seus cursos e vínculos.
GET	<code>/api/nosql/estudantes</code>	Lista todos os estudantes cadastrados..
GET	<code>/api/nosql/estudantes/{cpf}</code>	Busca um estudante específico pelo CPF.
PUT	<code>/api/nosql/estudantes/{cpf}</code>	Substitui todos os dados do estudante (atualização total).
DELETE	<code>/api/nosql/estudantes/{cpf}</code>	Remove um estudante pelo CPF.

6.4. Exemplo de Fluxo (Criação)

1. O cliente envia um JSON via `POST`.
2. O Controller recebe o `EstudanteRequestDTO` e aplica as validações (`@Valid`).
3. O Service converte o DTO para `EstudanteDocument`, verificando se o CPF ou Matrícula já existem (lançando `DuplicateKeyException` se sim).
4. O `EstudanteNoSqlRepository` salva o documento no MongoDB.
5. O documento salvo é convertido para `EstudanteResponseDTO` e retornado com status `201 Created`.

6.5. Tratamento de Exceções

Um `GlobalExceptionHandler` centraliza o tratamento de erros:

- **404 (Not Found):** `ResourceNotFoundException` – estudante não encontrado.
- **409 (Conflict):** `DuplicateKeyException` – CPF ou matrícula já cadastrados.
- **400 (Bad Request):** `MethodArgumentNotValidException` – erros de validação nos campos.

7. Testes e Evidências

7.1. Ambiente de Testes

Os testes foram realizados utilizando scripts em **PowerShell** que consomem a API REST. O ambiente de execução consistiu em:

- **Aplicação:** Rodando localmente na porta `8081`.
- **Banco de Dados:** MongoDB em uma máquina t3.micro da EC2
- **Ferramenta de visualização:** MongoDB Compass para inspeção em tempo real dos dados.

7.2. Script de Testes (test-nosql.ps1)

O script executa as quatro operações do CRUD de forma sequencial:

1. **Criação (POST):** Insere um estudante com CPF `33333333302`, matrícula `999001` e um curso com vínculo ativo.
2. **Leitura (GET all):** Lista todos os estudantes para confirmar a inserção.
3. **Busca por CPF (GET):** Recupera o estudante criado.
4. **Atualização (PUT):** Altera o MC (média) e o ano de ingresso.
5. **Deleção (DELETE):** Remove o estudante do banco.


```

=====
TESTE CRUD NOSQL
CPF: 3333333302 | Matricula: 999001
=====

Abra o MongoDB Compass na colecao 'estudantes'.
Pressione Enter para começar (certifique-se que a colecao esta vazia):

[1] CRIANDO ESTUDANTE...
SUCESSO: Criado (ID: 6a52f7e1ecc9db0fb7054516)
Nome: Joao NoSQL, MC: 85

Atualize o Compass (Refresh) e verifique.
Pressione Enter para continuar:

```

Figura 1: Execução do script PowerShell (POST)

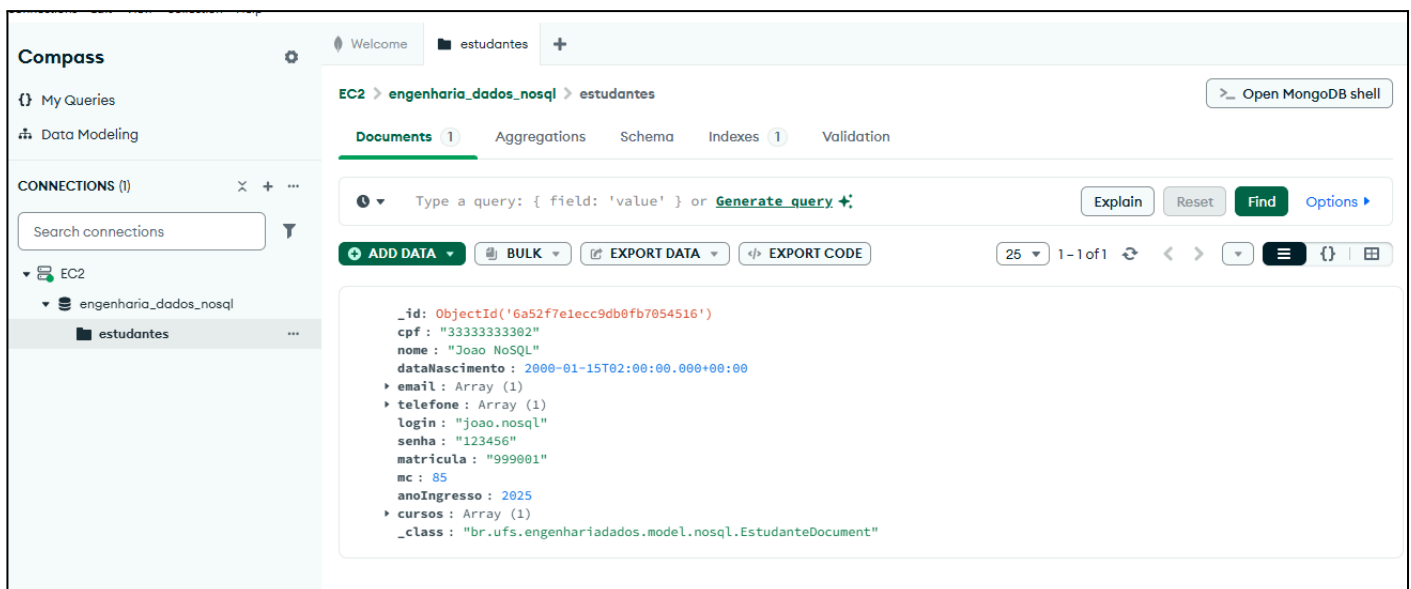


Figura 2: Monitoramento no Mongoddb Compass

```

Pressione Enter para continuar:

[2] LISTANDO TODOS...
Total: 1
[3] BUSCANDO POR CPF...
Encontrado: Nome Joao NoSQL, MC 85, Ano 2025
[4] ATUALIZANDO (MC=90, Ano=2026)...
SUCESSO: Atualizado! MC: 90 (antes 85), Ano: 2026 (antes 2025)

Atualize o Compass (Refresh) e verifique o documento atualizado.

```

Figura 3: Execução do script PowerShell (GET e PUT)

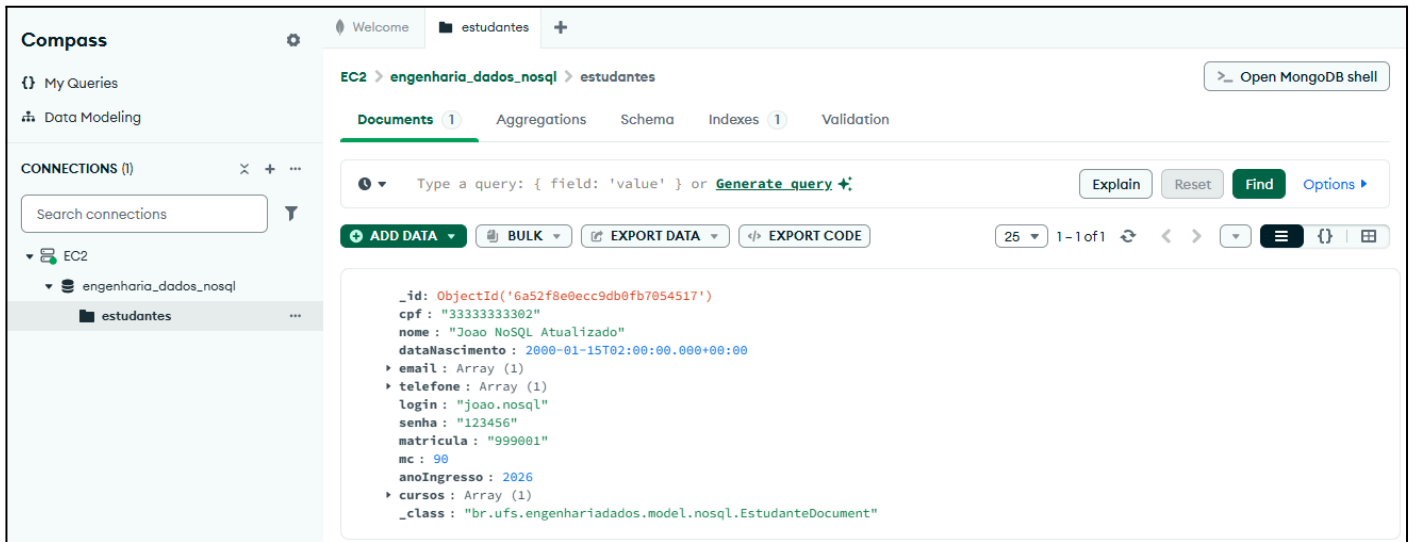


Figura 4: Monitoramento no Mongodb Compass

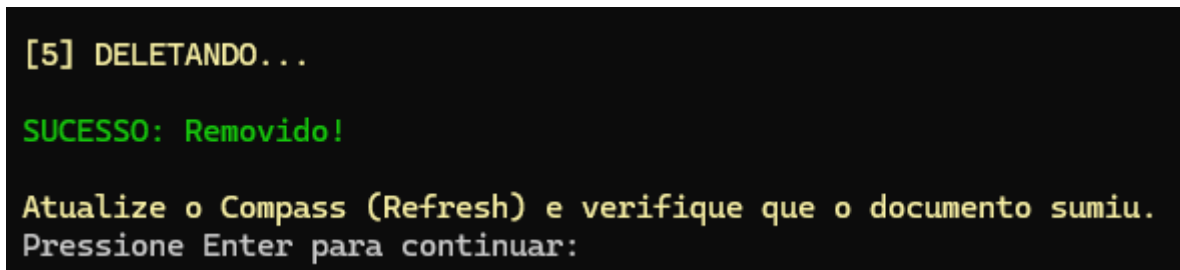


Figura 5: Execução do script PowerShell (DELETE)

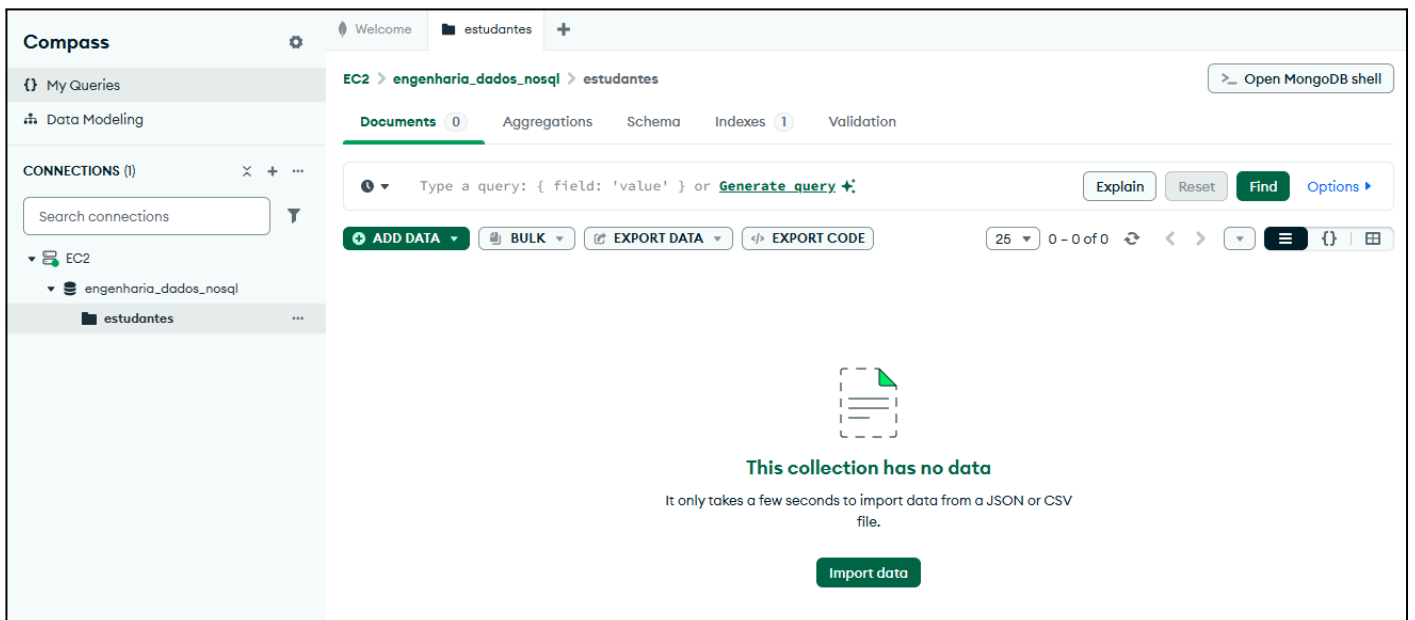


Figura 5: Monitoramento no Mongodb Compass

8. Conclusão

A Parte 2 do trabalho prático foi concluída com sucesso. Foi possível mapear o modelo relacional para o MongoDB utilizando uma abordagem desnormalizada, que se mostrou eficiente para o domínio proposto.

Os principais aprendizados obtidos foram:

- A importância de compreender o padrão de acesso aos dados para definir a modelagem correta (normalizado vs. desnormalizado).
- Como garantir restrições de integridade em um banco que não as possui nativamente, utilizando índices e validações na aplicação.
- As vantagens da flexibilidade de esquema do MongoDB para acomodar estruturas hierárquicas complexas (subdocumentos e arrays).
- A necessidade de configurar corretamente a serialização/deserialização JSON (profundidade e codificação UTF-8) ao integrar com clientes como o PowerShell.

Todas as operações de CRUD foram validadas e demonstradas, tanto via scripts automatizados quanto pela inspeção direta no banco de dados hospedado na AWS, atendendo integralmente aos requisitos da disciplina.

9. Conclusão

- MongoDB. (2026). *MongoDB Manual*. Disponível em: <https://docs.mongodb.com/>
- Spring. (2026). *Spring Data MongoDB - Reference Documentation*. Disponível em: <https://spring.io/projects/spring-data-mongodb>